Assignment: Neural Network (NN)

This assignment requires you to design, implement, tune, and compare two distinct Neural Network architectures—a **Shallow Neural Network** and a **Deep Neural Network**—within a single, fully automated machine learning workflow hosted on GitHub. You will evaluate their performance on a binary or multi-class tabular classification dataset of your choice (e.g., Diabetes, Heart Disease, or Bank Churn).

1. GitHub & Automation (Mandatory)

To ensure reproducibility and clean production standards, your submission must be fully automated.

- **Repository Setup:** Create a public GitHub repository.
- **Asset Hosting:** Upload your raw dataset (.csv) and your final Google Colab Notebook (.ipynb) directly to this repository.
- **Zero Manual Intervention:** Your Colab notebook must automatically fetch the dataset using its **raw GitHub URL**.
- **Runtime Constraint:** Manual file uploads via the Colab sidebar or local directory mapping are strictly forbidden. The notebook must execute from start to finish via Runtime -> Run all.

2. Pipeline Steps in Colab

Your notebook must follow a clean, sequential data science pipeline. Each step must be clearly delineated with markdown headers.

Data Preprocessing

1. Handle any missing data values using appropriate imputation strategies.
2. Encode categorical features using One-Hot or Ordinal encoding.
3. Split the data into stratified Train and Test sets (e.g., 80/20 split).

Shallow NN Implementation

1. Build a feedforward neural network using PyTorch.
2. **Configuration:** Must contain exactly one hidden layer.
3. **Hyperparameter Tuning:** Use a validation split to tune the number of hidden units, activation functions (e.g., ReLU, Sigmoid), and batch size.

Deep NN Implementation

1. Build a deep feedforward network using the same framework.
2. **Configuration:** Must contain at least three hidden layers.
3. **Regularization:** Incorporate regularization techniques (e.g., Dropout layers or L2 weight regularization) to combat overfitting.
4. **Hyperparameter Tuning:** Optimize the learning rate, selection of optimizer (e.g., Adam vs.

SGD), and the number of training epochs.

3. Required Visualizations (2x1 Comparison Matrix)

You are required to generate and display performance metrics on the **Test Set** for both optimized models. For direct comparison, outputs 1, 2, and 3 must be rendered as side-by-side plots using a **2x1 subplot matrix**.

1. Training History

- **Type:** Line graph
- **Comparison Format (Shallow vs. Deep):** 2x1 Matrix. Plot training vs. validation loss and accuracy across all epochs for both models to diagnose convergence.

2. Confusion Matrix

- **Type:** Heatmap
- **Comparison Format (Shallow vs. Deep):** 2x1 Matrix. Display the true positives, true negatives, false positives, and false negatives using a colored heatmap (seaborn.heatmap).

3. ROC Curve

- **Type:** Plot visualization
- **Comparison Format (Shallow vs. Deep):** 2x1 Matrix. Plot the Receiver Operating Characteristic (ROC) curve and explicitly print the Area Under the Curve (AUC) score for both models.

4. Evaluation Metrics

- **Type:** Bar Chart
- **Comparison Format (Shallow vs. Deep):** Combined Chart. A single, grouped bar chart comparing both models across: Accuracy, Precision, Recall, F1-Score, and AUC.

5. Network Structure

- **Type:** Visual Topology
- **Comparison Format (Shallow vs. Deep):** Sequential Output. A visual representation of both architectures (using keras.utils.plot_model, a clear text-based layer topology table, or a structured architectural summary).

4. Performance Interpretation & Analysis

Directly below your visualizations, include a dedicated Markdown cell containing a **3-5 sentence critical analysis** summarizing the results.

> **Analysis Guidance:** You must explicitly contrast the performance of the Shallow network versus the Deep network. Address whether the deeper architecture provided a justifiable lift in evaluation metrics, or if it suffered from overfitting based on the divergence observed in

your training history curves.

5. Submission

Provide the following two links in your final submission PDF:

- **GitHub Link:** URL to your public repository containing the dataset and your .ipynb file.
- **Colab Link:** The direct link to your live Google Colab notebook.
- **Dataset Link:** The direct link to the dataset.

Penalty: Failing to maintain any of these automation instructions will result in a penalty. If I have to manually upload files or debug file paths to make your code run, points will be deducted